

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



Sistemas Distribuidos

Laboratorio. S11

MexiBanco; una verdad, varias copias

Grupo: 02

Profesor: Oscar Manuel Ruiz Uscardo

Alumna: Segura Cedeño Luisa María

Semestre 2026-1

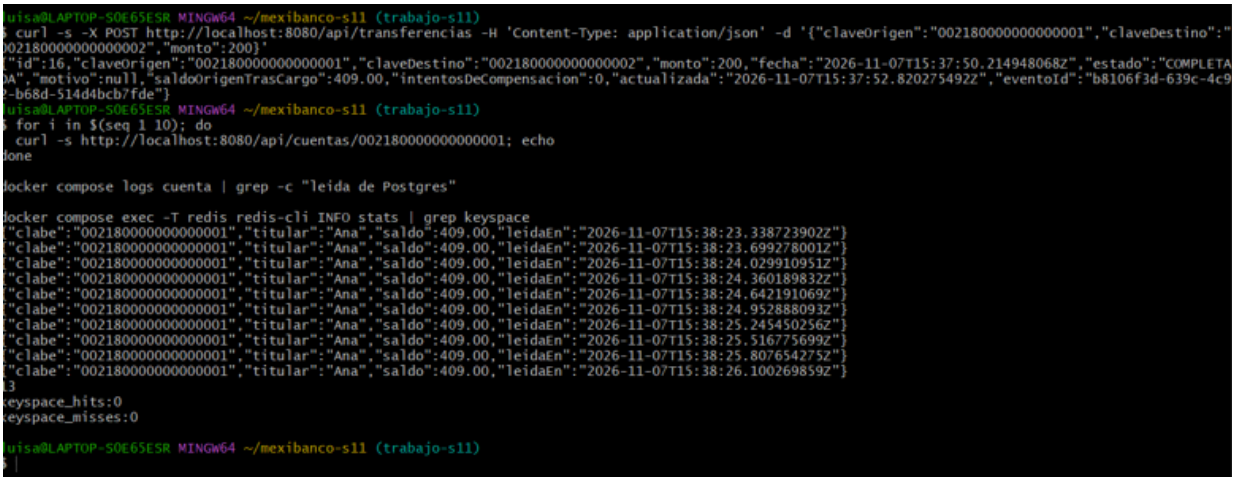
Fecha de Entrega: 07/11/2026

MexiBanco: una verdad, varias copias

1. El punto de partida

Consultas

Se solicitó 10 veces seguidas el saldo de Ana (GET /api/cuentas/002180000000000001, saldo 409.00) por el gateway, sin caché. Cada una de las 10 respuestas trae un leida en distinto, lo que muestra que cada consulta se resolvió leyendo la base de datos.



Lecturas a Postgres

Al contar en el log de cuenta las líneas "leida de Postgres" el resultado fue 13: las 10 consultas más las lecturas que hizo la transferencia de 200 pesos de Ana a Beto. En Redis, keyspace_hits:0 y keyspace_misses:0, es decir, nadie lo usa todavía.

Conclusión del punto de partida: diez consultas, diez lecturas a Postgres.

2. La caché

Código explicado

Se usó el patrón cache-aside, la aplicación pregunta primero a Redis y, si el saldo no está, lo lee de Postgres y deja una copia para la siguiente consulta. Se cambiaron tres archivos del servicio cuenta:

- ConfiguracionCache.java: Agregué @EnableCaching, en el cual @Cacheable y @CacheEvict se ignoran sin avisar. Además, el @Bean configura la caché "cuentas" con tres cosas; entryTtl(ttl), para que cada copia viva 30 segundos y luego Redis la borre sola; un Jackson2JsonRedisSerializer de CuentaVista, para guardar el saldo como JSON legible y no en binario de Java; y transactionAware(), para que el borrado de la copia espere al commit de la transacción.

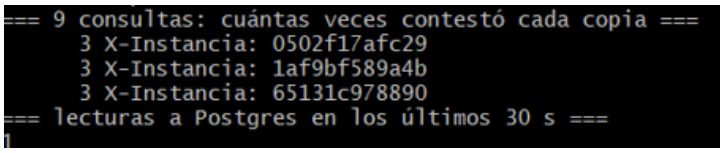
- ConsultaDeSaldo.java: Se puso @Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe") arriba del método consultar. Con eso, si la copia ya está en Redis contesta Redis y el método no corre; si no está, corre, lee Postgres y Spring guarda el resultado con una clave como cuentas::002180000000000001.

- CuentaService.java: Se puso @CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe") en cargar y en abonar, junto al @Transactional que ya tenían. Cuando cambia el saldo, se borra la copia (no se actualiza), y la siguiente consulta la vuelve a llenar con el saldo nuevo.

No se tocó el consultar de CuentaService; es la lectura que usa cargar para decidir si alcanza el saldo y siempre debe ir a Postgres. Si estuviera en caché, un cargo podría aprobarse con un saldo de 30 segundos. Por eso la consulta que se muestra vive en otra clase, ConsultaDeSaldo, y además así la anotación pasa por el proxy de Spring.

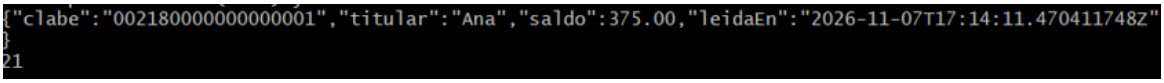
Tres copias compartiendo la caché

Después de reconstruir la cuenta (docker compose up --build -d cuenta), se hicieron 9 consultas del saldo de Ana por el gateway. Cada una de las tres copias de cuenta contestó 3 veces (tres X-Instancia distintos), pero en los últimos 30 segundos solo hubo 1 lectura a Postgres. Es decir, las tres copias comparten la misma caché en Redis: una sola copia guardó el saldo y las otras dos lo tomaron de ahí.



JSON Y TTL en Redis

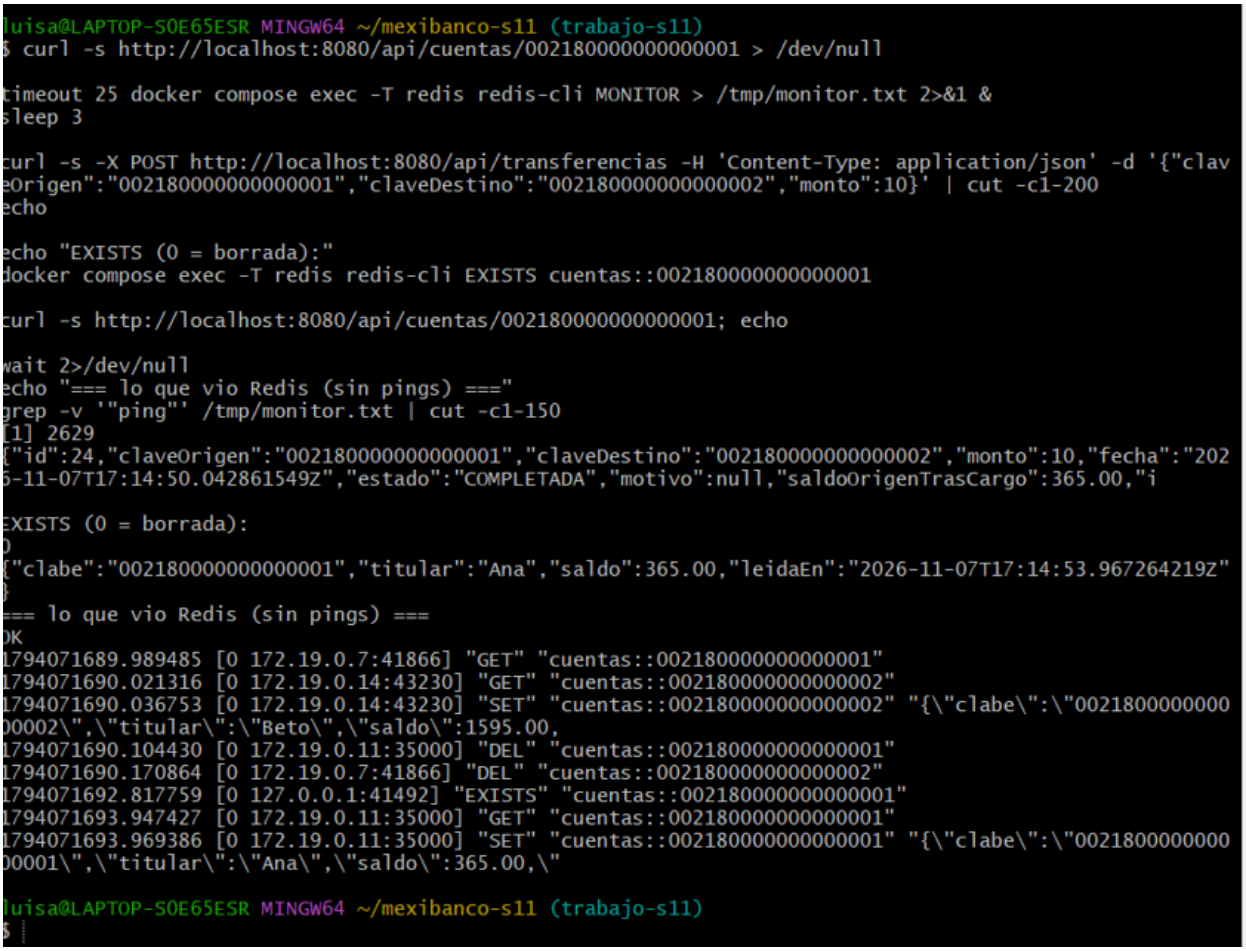
Con redis-cli GET cuentas::002180000000000001 se ve el valor guardado como JSON legible de CuentaVista (clabe, titular, saldo y leidaEn). Con TTL cuentas::002180000000000001 sale 21, o sea que a esa copia le quedaban 21 segundos de vida de los 30 configurados.



Copia borrada después de una transferencia

Con MONITOR corriendo en segundo plano, transferí 10 pesos de Ana a Beto (transferencia 24, COMPLETADA, saldo de Ana tras el cargo 365.00). En Redis aparece un GET de la saga, luego

un DEL de cuentas:: 001 después del commit del cargo y un DEL de cuentas:: 002 después del abono a Beto. EXISTS dio 0, así que la copia de Ana ya no estaba. Al pedir el saldo otra vez hubo un miss (GET), se leyó Postgres con un leida en nuevo y se guardó el saldo nuevo con un SET.



3. Tus medidas

Hit Ratio de A y B

El hit ratio es hits / (hits + misses). Antes de cada escenario se pusieron en cero los contadores con CONFIG RESETSTAT.

Escenario A: (100 consultas seguidas del saldo de Ana): keyspace_hits = 99 y keyspace_misses = 1, es decir 99 / 100 = 99 %. El único miss fue la primera consulta, que llenó la caché; las otras 99 salieron de Redis.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexibanco-s11 (trabajo-s11)
$ cd ~/mexibanco-s11
docker compose exec -T redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
done
echo "=== ESCENARIO A ==="
docker compose exec -T redis redis-cli INFO stats | grep keyspace
OK
=== ESCENARIO A ===
keyspace_hits:99
keyspace_misses:1
```

Escenario B: Lo mismo, pero con una transferencia de 1 peso de Ana a Beto cada 10 consultas. keyspace_hits = 100 y keyspace_misses = 20, es decir 100 / 120 = 83.3 %. Baja porque cada transferencia borra la copia de Ana y la de Beto (DEL después del commit); en las 10 transferencias que hice, la siguiente lectura de cada cuenta fue un miss. Los contadores de Redis también incluyen los GET que hacen la saga de transferencia, por eso los misses no son solo de mis consultas.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexibanco-s11 (trabajo-s11)
$ cd ~/mexibanco-s11
docker compose exec -T redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application
/json' -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
  fi
done
echo "=== ESCENARIO B ==="
docker compose exec -T redis redis-cli INFO stats | grep keyspace
OK
=== ESCENARIO B ===
keyspace_hits:100
keyspace_misses:20

luisa@LAPTOP-S0E65ESR MINGW64 ~/mexibanco-s11 (trabajo-s11)
$
```

Latencia con y sin caché

Se midieron 100 consultas seguidas del saldo de Ana, tres veces con caché y tres veces sin ella (CACHE_TIPO=none), y se calculó promedio, mediana y p95 en milisegundos. A continuación, muestro todas las corrida:

Caso	Promedio	Mediana	p95	Lecturas a Postgres
Con caché, corrida 1	38.83 ms	20.53 ms	58.58 ms	1
Con caché, corrida 2	20.92 ms	18.64 ms	33.05 ms	1
Con caché, corrida 3	22.63 ms	18.10 ms	36.52 ms	1

Sin caché, corrida 1	36.49 ms	21.93 ms	46.95 ms	100
Sin caché, corrida 2	247.34 ms	19.50 ms	56.35 ms	100
Sin caché, corrida 3	19.70 ms	17.82 ms	28.60 ms	100

Con caché:

```
##### CON CACHE - corrida 1 #####
promedio=38.83 ms  mediana=20.53 ms  p95=58.58 ms
Lecturas a Postgres durante esta corrida: 1
##### CON CACHE - corrida 2 #####
promedio=20.92 ms  mediana=18.64 ms  p95=33.05 ms
Lecturas a Postgres durante esta corrida: 1
##### CON CACHE - corrida 3 #####
promedio=22.63 ms  mediana=18.10 ms  p95=36.52 ms
Lecturas a Postgres durante esta corrida: 1
```

Sin caché:

```
$ bash ~/medir.sh "SIN CACHE - corrida 1"
sleep 15
bash ~/medir.sh "SIN CACHE - corrida 2"
sleep 15
bash ~/medir.sh "SIN CACHE - corrida 3"
##### SIN CACHE - corrida 1 #####
promedio=36.49 ms  mediana=21.93 ms  p95=46.95 ms
Lecturas a Postgres durante esta corrida: 100
##### SIN CACHE - corrida 2 #####
promedio=247.34 ms  mediana=19.50 ms  p95=56.35 ms
Lecturas a Postgres durante esta corrida: 100
##### SIN CACHE - corrida 3 #####
promedio=19.70 ms  mediana=17.82 ms  p95=28.60 ms
Lecturas a Postgres durante esta corrida: 100
```

En mi computadora la caché no hizo la consulta más rápida. Las medianas son casi iguales (18 a 21 ms con caché, 18 a 22 ms sin caché) y los p95 se traslapan entre casos. La corrida 2 sin caché tiene un promedio de 247 ms, pero su mediana es 19.5 ms: fue por unas pocas consultas muy lentas, probablemente porque la computadora estaba cargada, y no por la ausencia de caché. Lo más probable es que el tiempo esté dominado por el gateway, la red de Docker y curl, y no por la lectura a Postgres, que es de una sola fila por clave y local.

Lecturas a postgres

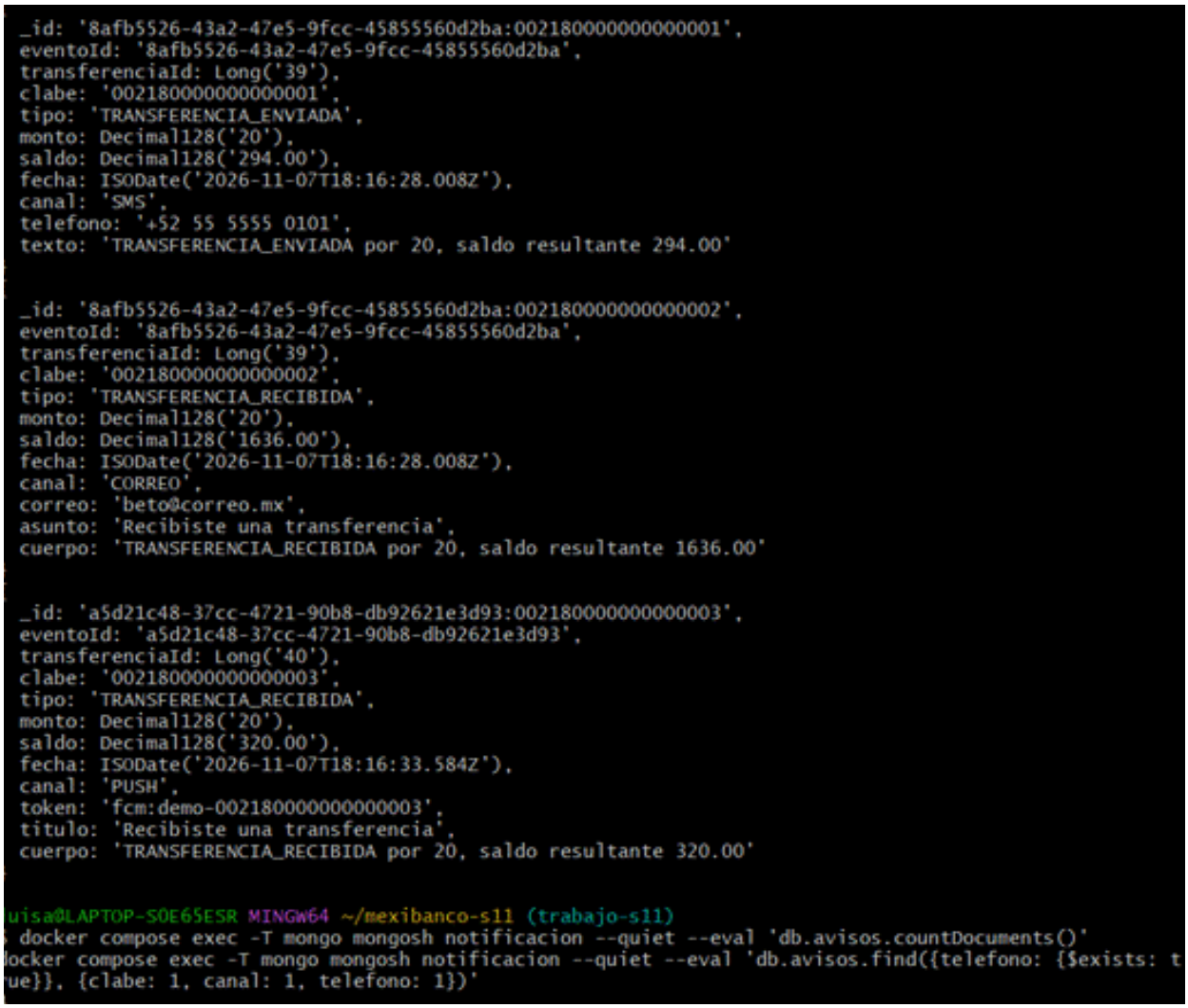
Con caché hubo 1 lectura a Postgres por cada 100 consultas (la primera de cada corrida, que llena la copia de 30 s). Sin caché hubo 100 de 100, una por cada consulta. Es decir, la caché redujo las lecturas a Postgres de 100 a 1 por corrida, una reducción de 99 %.

4. Los avisos

Tres formas en mogosh

Después de reconstruir la notificación se hicieron dos transferencias de 20 pesos (de Ana a Beto y de Ana a Carla) y la colección avisos pasó de 0 a 4 documentos, dos por transferencia. Cada aviso tiene los campos comunes (_id = eventoId:clabe, eventoId, transferenciaId, clabe, tipo, monto y saldo como Decimal128, fecha y canal) más los de su canal:

SMS (Ana): telefono y texto. CORREO (Beto): correo, asunto y cuerpo. PUSH (Carla): token, título y cuerpo. Los tres conviven en la misma colección sin ALTER TABLE ni columnas vacías.



El filtro por teléfono (db.avisos.find({telefono: {$exists: true}}, ...)) trae solo los 2 avisos de Ana, que son los únicos documentos que tienen ese campo: el esquema lo aplica quien lee.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval 'db.avisos.countDocuments()'
docker compose exec -T mongo mongosh notificacion --quiet --eval 'db.avisos.find({telefono: {$exists: t
rue}}, {clabe: 1, canal: 1, telefono: 1})'
4
[
  {
    _id: '8afb5526-43a2-47e5-9fcc-45855560d2ba:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'a5d21c48-37cc-4721-90b8-db92621e3d93:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Endpoint con 10

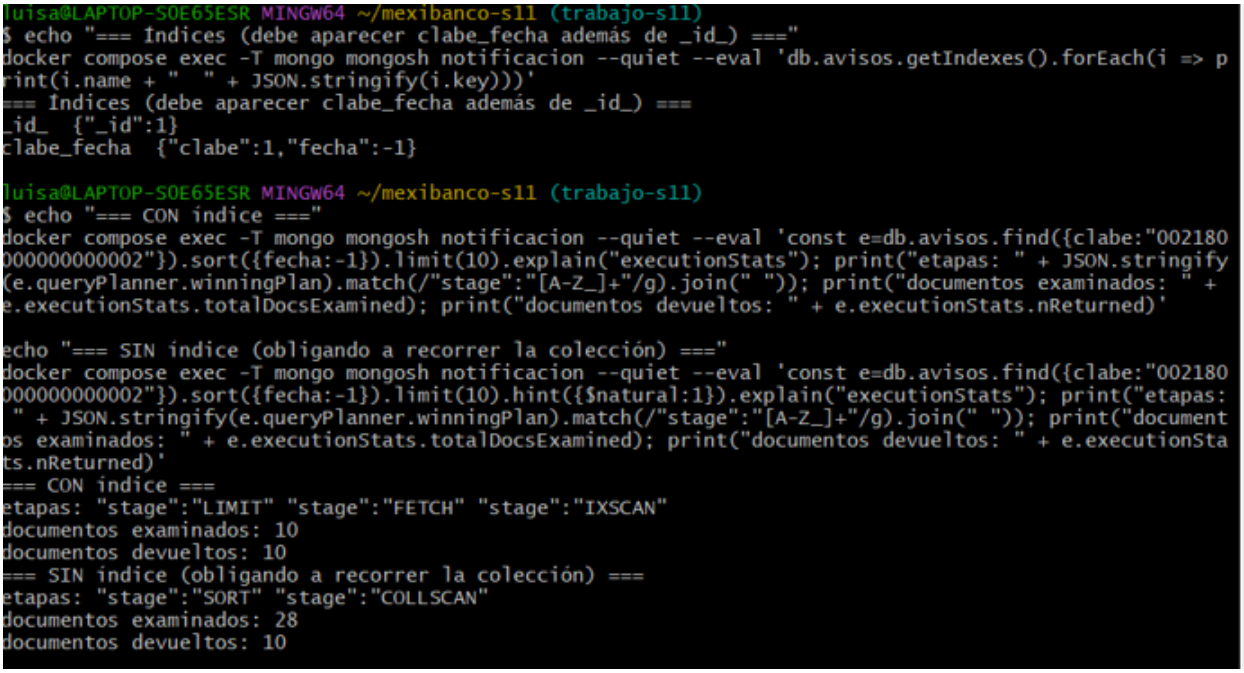
Se hicieron doce transferencias de 1 peso de Ana a Beto (todas respondieron 201) y pedí la bitácora de Beto por el gateway: GET /api/notificaciones/002180000000000002. Devolvió exactamente 10 avisos, todos de canal CORREO, ordenados del más reciente (18:21:34) al más viejo (18:21:17). Beto tiene más de 10 avisos guardados, pero el endpoint solo regresa los últimos 10 por el limit(ULTIMOS) de la consulta.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexibanco-s11 (trabajo-s11)
$ cd ~/mexibanco-s11
for i in $(seq 1 12); do
  curl -s -o /dev/null -w "transferencia $i: código %{http_code}\n" -X POST http://localhost:8080/api/t
ransferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180000000000001","claveDestino
":"002180000000000002","monto":1}'
  sleep 1
done
sleep 5
transferencia 1: código 201
transferencia 2: código 201
transferencia 3: código 201
transferencia 4: código 201
transferencia 5: código 201
transferencia 6: código 201
transferencia 7: código 201
transferencia 8: código 201
transferencia 9: código 201
transferencia 10: código 201
transferencia 11: código 201
transferencia 12: código 201

luisa@LAPTOP-S0E65ESR MINGW64 ~/mexibanco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/notificaciones/002180000000000002 > /tmp/beto.json
echo "Avisos devueltos (debe ser 10):"
grep -o '"canal"' /tmp/beto.json | wc -l
echo "Canales:"
grep -o '"canal":"[A-Z]*"' /tmp/beto.json | sort | uniq -c
echo "Fechas, de la más nueva a la más vieja:"
grep -o '"fecha":"[^"]*"' /tmp/beto.json
Avisos devueltos (debe ser 10):
10
Canales:
     10 "canal":"CORREO"
Fechas, de la más nueva a la más vieja:
"fecha":"2026-11-07T18:21:34.989+00:00"
"fecha":"2026-11-07T18:21:32.901+00:00"
"fecha":"2026-11-07T18:21:31.205+00:00"
"fecha":"2026-11-07T18:21:29.340+00:00"
"fecha":"2026-11-07T18:21:27.566+00:00"
"fecha":"2026-11-07T18:21:25.946+00:00"
"fecha":"2026-11-07T18:21:24.339+00:00"
"fecha":"2026-11-07T18:21:22.536+00:00"
"fecha":"2026-11-07T18:21:19.054+00:00"
"fecha":"2026-11-07T18:21:17.008+00:00"
```

Índice

Al arrancar, crearIndice() creó el índice clabe_fecha con clabe ascendente y fecha descendente, que sigue la forma de la consulta de ultimos(): primero se filtra por clabe y luego se ordena por fecha. En getIndexes() aparece junto al índice _id_ que Mongo crea siempre.



Los dos explain

Para que la salida cupiera en una captura, imprimí solo un resumen del explain("executionStats") (etapas del plan, documentos examinados y devueltos) y no el JSON completo. La consulta es la misma de ultimos(): clabe de Beto, ordenada por fecha descendente, limit 10.

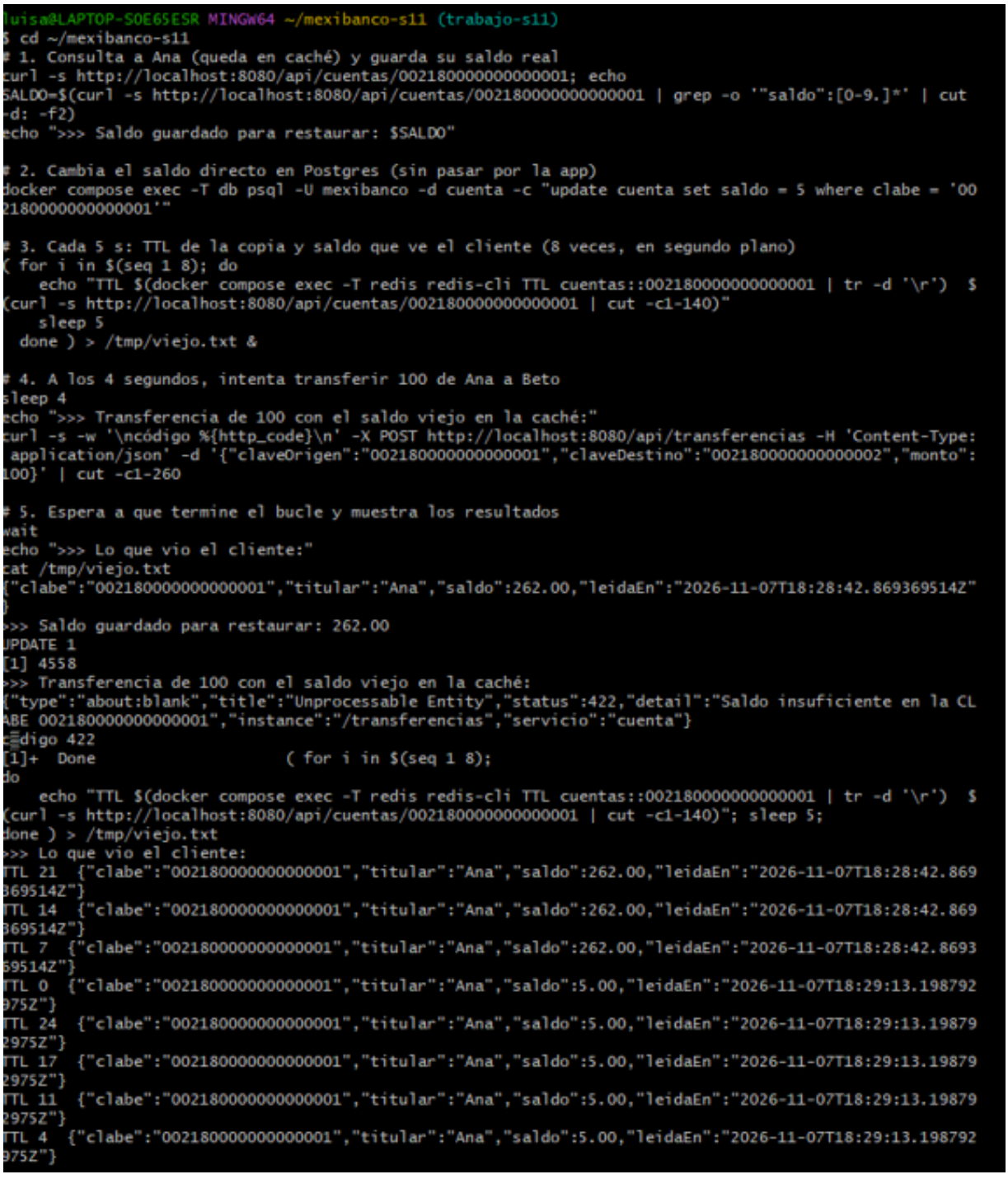
Con índice: Etapas LIMIT, FETCH e IXSCAN; examinó 10 documentos y devolvió 10. Sin índice (forzando hint({$natural: 1})): etapas SORT y COLLSCAN; examinó 28 documentos, todos los de la colección, y devolvió 10. Con 28 avisos la diferencia es chica, pero con millones de avisos sería la diferencia entre leer 10 y leerlos todos, además de ordenar en memoria.

Mi predicción: La app le sigue mostrando a Ana su saldo de antes, pero la transferencia de 100 no pasa, porque el cargo lee Postgres y no la caché.

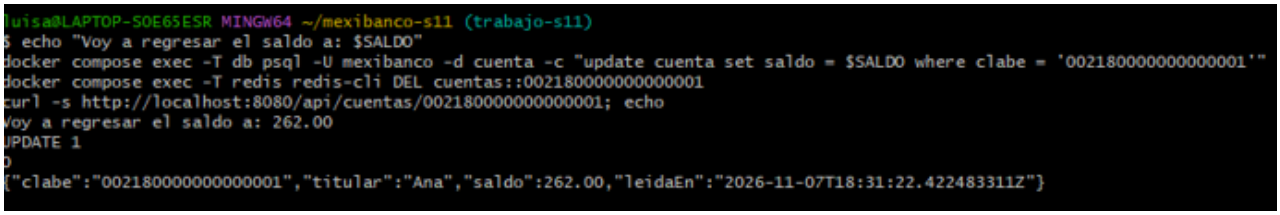
Consulté a Ana (saldo 262.00, quedó en la caché) y cambié su saldo directo en Postgres con un UPDATE a 5, sin pasar por la aplicación, así que ningún @CacheEvict borró la copia. Cada 5 segundos medí el TTL y el saldo que ve el cliente: con TTL 21, 14 y 7 siguió viendo 262.00; cuando el TTL llegó a 0 la copia caducó, se leyó Postgres y apareció 5.00 con un leida en nuevo. Es decir, el cliente vio un saldo que ya no existía durante unos 30 segundos, el TTL completo (la

copia se creó a las 18:28:42 y se releyó a las 18:29:13).

Mientras el saldo viejo seguía en la caché intenté transferir 100 de Ana a Beto. Respondió 422 Unprocessable Entity: "Saldo insuficiente en la CLABE 002180000000000001". El cargo leía Postgres, donde Ana tenía 5, y no la caché, que decía 262. La caché muestra, nunca decide.



Al terminar regresé el saldo de Ana a 262.00 con otro UPDATE (el de antes de la prueba) y borré su copia de Redis para que no quedara el 5 viejo. La consulta final devolvió 262.00. Este UPDATE directo rompió a propósito el cuadre entre el saldo y los movimientos, y el segundo lo dejó como estaba.



5. Preguntas de cierre

- **¿Por qué MongoDB para los avisos y no para el saldo? Usa lo que viste: la forma de cada aviso, cómo se leen y qué tendría que garantizar el saldo.**

  Porque los avisos y el saldo son datos muy diferentes. Cada aviso tiene una forma diferente según su canal: el SMS lleva teléfono y texto, el correo lleva correo, asunto y cuerpo, y el push lleva token, título y cuerpo. En mongosh vi los tres tipos en la misma colección, sin columnas vacías y sin ALTER TABLE. Además se escriben una sola vez, no se actualizan, se leen enteros por CLABE (el endpoint de los últimos 10 con el índice clabe_fecha) y no hay relaciones ni reglas entre avisos.

  El saldo es dinero, tiene que cambiar con una transacción local y control de versión, cuadrar con los movimientos y no perderse ni contradecirse. Si estuviera en dos bases sin una transacción común, una podría quedarse sin el dato. Por eso el saldo se queda en Postgres, que es la verdad, y el resto son copias o bitácoras.

- **¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?**

  En el escenario A (100 consultas seguidas) medí 99 hits y 1 miss, un hit ratio de 99 %. En el escenario B (una transferencia cada 10 consultas) medí 100 hits y 20 misses, 83.3 %. Baja porque cada transferencia borra la copia de Ana y la de Beto, y la siguiente lectura de cada una es un miss que vuelve a llenar la caché.

  Con mis números de latencia, las medianas fueron casi iguales con caché (18 a 21 ms) y sin ella (18 a 22 ms), así que en mi máquina no se vió una gran velocidad. Lo que sí cambió fue la carga: con caché hubo 1 lectura a Postgres por corrida de 100 consultas y sin caché hubo 100.

- **¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la Parte 4.**

  Podría aprobar un cargo con un saldo que ya no existe. En la Parte 4 la caché mostró 262.00 durante unos 30 segundos mientras Postgres tenía 5. Si el cargo hubiera leído la caché, la transferencia de 100 se habría aprobado y Ana habría quedado sobregirada, con dinero que no tenía. Como el cargo leía Postgres, la transferencia respondió 422 por saldo insuficiente.

- **Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?**

  Se hace DEL y no SET porque con SET habría que calcular y escribir el valor correcto en Redis, y dos cambios seguidos podrían dejar guardado el saldo equivocado si sus escrituras llegan en otro orden. Con DEL no se guarda ningún valor: la siguiente consulta vuelve a leer Postgres, que es la verdad, y llena la caché con el saldo nuevo. En el MONITOR se vio el DEL, luego el GET (miss) y el SET con el saldo nuevo.

  Se hace después del commit y no antes porque, si se borrara antes, otra consulta podría leer en esa ventana el saldo viejo de Postgres (el cambio aún no está confirmado) y volver a guardarlo en la caché, donde se quedaría hasta 30 segundos. Con transactionAware() el DEL espera al commit, así que lo que se vuelve a leer ya es el saldo nuevo.